

Subqueries or Nested queries

Subqueries

```
- select *  
  from HomePro.Customers  
 where age >  
        (select avg(age)  
         from HomePro.Customers)
```

Outer query

Inner query

SubQuery

Type subqueries by returns

Single-valued

- Query return a single value

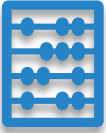
Multivalued

- Query returns multiple values as a single column

Table-valued

- Query returns a row set with multiple columns

Scalar value subquery

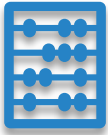


Business Case:

Get customers that are older than average customer age

```
select * from HomePro.Customers
where age >
    (select avg(age) from HomePro.Customers)
```

Scalar value subquery

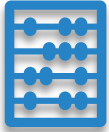


Business Case:

Get customers that have quote less than average quote

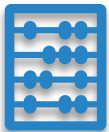
```
select * from HomePro.Customers C
  join HomePro.Quotes q
    on c.CustomerId = q.CustomerId
where q.Estimation <
      (select avg(Estimation) from HomePro.Quotes)
```

Multivalued subqueries



Get customers that have scheduled work

```
select * from HomePro.Customers
where CustomerId in
    (select CustomerId from HomePro.Schedules)
```



Get customers that do NOT HAVE scheduled work

```
select * from HomePro.Customers
where CustomerId not in
    (select CustomerId from HomePro.Schedules)
```

Table valued subqueries

1. Find all combination of Schedules with Quotes
2. Use the subquery result in outer query

```
select * from(  
  select  
    s.CustomerId, s.JobType,  
    Q.Description as QuoteDescription, q.Estimation  
  from HomePro.Schedules S  
    join HomePro.Quotes q on s.CustomerId = q.CustomerId  
) as subquery
```

Table valued subqueries (Example 1)

1. Find all combination of Schedules with Quotes
2. Use the subquery result joining with Customers

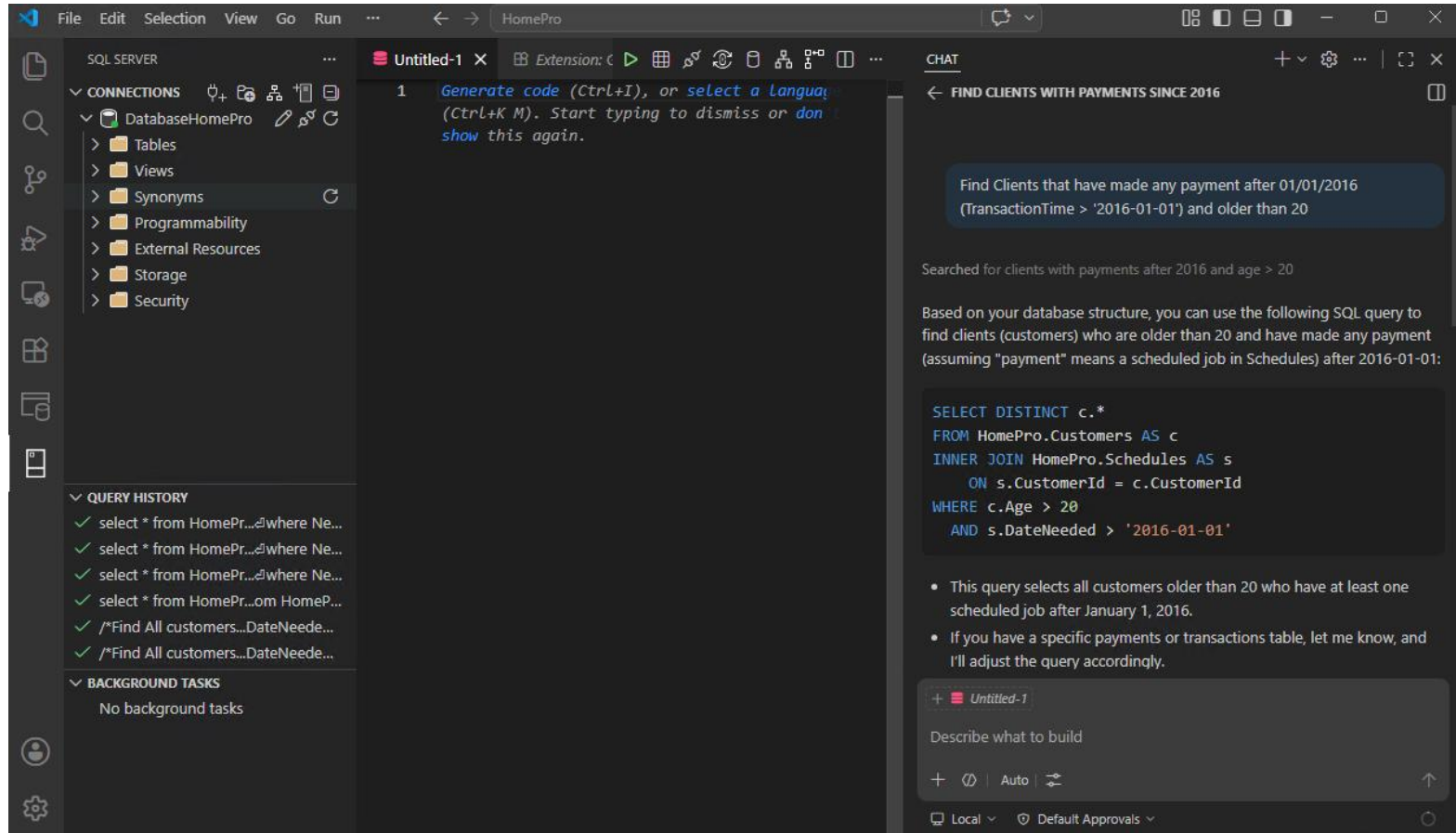
```
select *  
From HomePro.Customers C  
join (  
    select s.CustomerId, s.JobType,  
           Q.Description as QuoteDescription, q.Estimation  
    from HomePro.Schedules S  
         join HomePro.Quotes q on s.CustomerId = q.CustomerId  
    ) as subquery  
on c.CustomerId = subquery.customerId
```


Table valued subqueries (Example 2)

1. Find Customers that older than 20
2. Use the subquery result to show Customers with Schedules using left join

```
select *  
from (  
    select * from HomePro.Customers  
    where Age > 20  
    ) subquery  
left join HomePro.Schedules S  
on subquery.CustomerId = s.CustomerId
```

GitHub Copilot in VS Code



Benefits / Pros

Increases Productivity

- Autocompletes.
- Reduces time spent on repetitive tasks
- Helps to focus more on problem-solving than syntax

Accelerates Learning

- Suggests idiomatic code and best practices in real time.
- Helps beginners learn new languages and frameworks by example.
- Acts as an “assistant” explaining patterns through code suggestions.

Supports Multiple Languages & Frameworks

Works with Python, JavaScript, C#, SQL, etc.

Caveats / Cons

Risk of Over-Reliance

- Students may rely on suggestions without understanding the logic.
- Can harm fundamental learning if used as a shortcut.

Potential for Incorrect or Suboptimal Code

- Suggestions are not guaranteed to be correct or efficient.
- May introduce bugs, security issues, or outdated practices.
- Requires careful review and testing.

Limited Understanding of Full Context.

Doesn't fully understand system architecture or business logic.
Can produce code that looks correct but doesn't fit real requirements.
Still requires human design decisions and validation.

Trust but Verify



How to Open / Enable Copilot

Step 1: Install the Copilot Extension

- Open Visual Studio Code
- Go to Extensions (left sidebar or press Ctrl + Shift + X)
- Search for: GitHub Copilot
- Click Install

Step 2: Sign In to GitHub

- After installation, VS Code will ask you to sign in
- Use your GitHub or Google account

Step 3: Open Copilot Chat (if enabled)

- Look at the sidebar
- Click the Copilot/Chat icon
- Or press: Ctrl + Alt + I (or Ctrl + Shift + I)
- Ask questions like:
 - “Generate SQL query for customers table”